Assignment 4 MIPS Game Project

Tic-Tac-Toe and Blackjack and Hangman in MIPS assembly

Dylan Flores. SDSU Fall 2026. October 18 2026

How to run the games

Each game is one file. tictactoe.asm and blackjack.asm and hangman.asm. They all use the same Bitmap Display settings. All of the code and the Java versions and the display images are on my GitHub at github.com/dflo042/cs/tree/main/mips-game-project

- Open the file in MARS 4.5 and assemble it with F3.
- Go to Tools and open Bitmap Display. Set Unit Width 4 and Unit Height 4 and Display Width 256 and Display Height 256 and Base address 0x10008000 ($gp). Click Connect to MIPS.
- Run with F5. Prompts show up in the Run I/O box at the bottom and the game is drawn in the Bitmap Display window. The board or cards are also printed as text in the console.
- Type an answer and press Enter. Bad input gives an error message and asks again. N at the play again prompt quits.

For the Java versions run javac TicTacToe.java Blackjack.java Hangman.java and then java TicTacToe or java Blackjack or java Hangman. They take the same input and print a text version of the display after each move.

Tic-Tac-Toe wants a cell number 1 to 9. Blackjack wants H or S in either case. Hangman wants one letter. All three ask Y or N to play again. Anything else is rejected with a message and nothing changes.

How the display and input work

I wrote one small set of graphics and input procedures and put the same copy in each file so every file assembles on its own. The display is 64 by 64 units with one word per unit holding a color as 0x00RRGGBB. That is 4096 words starting at 0x10008000. The address of a pixel is 0x10008000 + (y * 64 + x) * 4. draw_pixel does that with two shifts and an add. It checks that x and y are inside 0 to 63 first so a bad coordinate can never overwrite anything else.

- fill_screen(color) clears the screen. draw_rect(x y w h color) is a filled rectangle. It has five arguments so the color is passed on the stack and the procedure reads it with lw $t9 0($sp) before saving its own registers.
- draw_glyph(x y char color) draws one character from a 3 by 5 font I made. The font is 5 bytes per character with bit 2 as the left pixel. It has 0-9 and A-Z and a few symbols. draw_text draws a string 4 pixels per character so 15 fit across.
- read_line uses syscall 8 to read a whole line and strips the newline. parse_int in Tic-Tac-Toe turns the text into a number and reports whether it really was one. I did this instead of syscall 5 because syscall 5 throws an error in MARS if you type letters. to_upper lets lower case letters work.

For conventions I used $a0 to $a3 for arguments and $v0 and $v1 for return values. $t registers are temporary and never expected to survive a jal. Loops that only call draw_pixel keep a counter in $t8 since draw_pixel only uses $t0 and $t1. Every procedure that uses $s registers pushes them and $ra on the stack first and pops them at the end. Leaf procedures do not touch the stack. Game state is in .data

as byte arrays and word counters. .eqv is used for colors and sizes and a macro for the print syscalls.

Game 1 Tic-Tac-Toe

Two players take turns typing a cell number. The board is a 9 byte array where 0 is empty and 1 is X and 2 is O. The eight winning lines are a 24 byte table of cell indexes so check_win is one loop over that table. After each move the program redraws that cell and runs check_win for the current player. It checks move_count for a draw at 9 and switches players with $s0 = 3 - $s0. Empty cells show a dim number so you know what to type. X is two thick diagonals and O is a ring. When someone wins highlight_line redraws the three winning marks in green and the strip at the bottom says X WINS! or O WINS! or DRAW.

Procedures are main for the game loop and prompt and validation and play again. reset_game. check_win(player). highlight_line(line player). print_console_board and print_cell. show_turn_status and show_status. draw_board_screen. cell_origin(index) which uses div and mflo and mfhi. draw_mark and draw_mark_color and draw_x_mark and draw_o_mark. Plus read_line and parse_int and the graphics procedures.

T1 X wins down the middle column

T2 O wins on the diagonal

T3 full board and a draw

Tests

- **T1** X wins with the middle column. I typed 5 1 9 3 2 7 4 6 8 N and it passed.
- **T2** O wins on the diagonal. I typed 1 3 2 5 4 7 N and it passed.
- **T3** Full board with no winner. I typed 1 2 3 5 4 6 8 7 9 N and it passed.
- **T4** (edge case) Bad input. Text then a blank line then 0 then 10 then spaces around a number then a taken cell. I typed abc (blank) 0 10 ' 5 ' 5 1 9 3 2 7 4 6 8 N and it passed.
- **T5** (edge case) Bad play again answer and the board resets for a second game. I typed 1 4 2 5 3 maybe y 1 2 3 5 4 6 8 7 9 n and it passed.

T4 log with bad input then a taken cell (what I typed is after each prompt)

```
Player X, choose a cell (1-9): abc
  ! That is not a whole number. Enter a digit from 1 to 9.
Player X, choose a cell (1-9):
  ! That is not a whole number. Enter a digit from 1 to 9.
Player X, choose a cell (1-9): 0
  ! Out of range. Enter a digit from 1 to 9.
Player X, choose a cell (1-9): 10
  ! Out of range. Enter a digit from 1 to 9.
Player X, choose a cell (1-9):  5

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
   ...
```

T3 log at the end of the draw game

```
Player X, choose a cell (1-9): 9

 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | X


*** It's a draw. ***
Play again? (Y/N): N
Thanks for playing!
```

Game 2 Blackjack

You play against a dealer that hits on 16 or less and stands on 17 or more. The deck is a 52 byte array holding 0 to 51 where rank is card mod 13 and suit is card / 13. Every round refills and shuffles it with a Fisher-Yates loop using syscall 42. The generator is seeded once from the clock with syscalls 30 and 40. deal_card appends the next card to a hand array and bumps its counter. hand_value counts aces as 11 and then subtracts 10 per ace while the total is over 21 so A+6 is 17 and A+6+9 is 16. A natural blackjack is checked first. 21 stands automatically and a bust ends the round. Otherwise the dealer flips the hidden card and draws to 17. Wins and losses and pushes are counted across rounds. Cards on the display are 9 by 12 with the rank and a suit letter in red for hearts and diamonds. The dealer's second card is a blue card back until you stand.

Procedures are main and shuffle_deck and deal_card(hand count pointer) and card_value and hand_value and to_upper. print_card and print_hand and print_player_line and print_table_hidden and print_dealer_full for the console. redraw_table(hide dealer) and draw_hand and draw_card and rank_char and draw_scores and draw_record and draw_number for the display. int_to_str pushes digits on the stack to reverse them.

My turn with the dealer card hidden

Round over. 19 vs 18 and I won

After the 40 round run

Tests

- **B1** Stand right away and let the dealer play. I typed S N and it passed.
- **B2** Basic strategy. Hit under 17 and stand at 17 or more. I typed H while under 17 then S then N and it passed.
- **B3** Three rounds in a row and the win loss push counters add up. I typed basic strategy Y Y N and it passed.
- **B4** (edge case) Bad commands (7 then a blank line then ?) are rejected without dealing a card. I typed 7 (blank) ? S N and it passed.
- **B5** (edge case) Hit until bust and then a bad play again answer. I typed H over and over then maybe then N and it passed.

The cards are random so every run is different. I also ran 40 rounds with a script that hits under 17 (B6). A checker recomputed every printed hand total from the card names and checked the dealer's 17 rule and the announced result. It checked 113 totals over 40 rounds with no mistakes. The final record was 18 won 19 lost 3 pushed.

B3 log showing one round of the basic strategy run

```
---------------- NEW ROUND ----------------
Dealer shows : AC  [??]
Your hand    : 8D 9H    (total 17)
(H)it or (S)tand? S
Dealer reveals: AC 6H
Dealer hand  : AC 6H    (total 17)

*** Push - it's a tie. ***
Record: 0 won, 0 lost, 1 pushed.
Play another round? (Y/N): Y

---------------- NEW ROUND ----------------
   ...
```

B4 log where bad commands are rejected

```
---------------- NEW ROUND ----------------
Dealer shows : JS  [??]
Your hand    : 8H 6D    (total 14)
(H)it or (S)tand? 7
  ! Please type H to hit or S to stand.
(H)it or (S)tand?
  ! Please type H to hit or S to stand.
(H)it or (S)tand? ?
  ! Please type H to hit or S to stand.
(H)it or (S)tand? S
Dealer reveals: JS QS
Dealer hand  : JS QS    (total 20)

*** Dealer wins. ***
   ...
```

Game 3 Hangman (my custom game)

I picked Hangman because it uses the requirements differently from the other two. There is a table of pointers to strings for the word list and a 26 entry array indexed by letter minus 'A' for the letters tried. The input is checked character by character and the drawing grows one piece at a time. The word is picked with syscall 42 from twelve computer words (COMPUTER ASSEMBLY REGISTER PROCEDURE MEMORY BRANCH STACK SYSCALL PIPELINE COMPILER HARDWARE PROGRAM). A guess is accepted only if it is exactly one character and a letter and not tried yet. A repeat does not cost a life. count_in_word gives how many times the letter appears and word_solved checks the word against the guessed array. Six misses loses and the word is shown with the missing letters in red. draw_figure(n) draws the first n parts of the person from the wrong guess counter so the scene is rebuilt from the state every redraw.

Procedures are main and reset_game and count_in_word(letter) and word_solved and to_upper and print_state. draw_scene calls draw_lives and draw_gallows and draw_figure(parts) and draw_word_row(reveal all) and draw_miss_row. draw_diag(x y dx) draws the arms and legs. draw_number and int_to_str draw the lives count.

H1 solved with no misses

H2 solved after three misses

H3 six misses and the word revealed

Tests

- **H1** Solve the word with no misses. I typed letters picked from the word list then N and it passed.
- **H2** Win after three misses so part of the figure is drawn. I typed Z Q J then letters from the word list then N and it passed.
- **H3** Six misses so the game is lost and the word is shown. I typed F J Q V X Z N and it passed.

- **H4** (edge case) Bad guesses. Two letters then a blank line then a digit then a symbol then a repeat (no life lost). I typed ab (blank) 1 ? e E F J Q V X Z N and it passed.
- **H5** (edge case) Lower case works and a bad play again answer and a new word with 6 lives on replay. I typed f j q v x z xyz Y F J Q V X Z n and it passed.

The word is random so for H1 and H2 a small script picked the letter that appears in the most words still matching the pattern. The losing tests use F J Q V X and Z because none of the twelve words have those letters.

H4 log with bad guesses and a repeat

```
Guess a letter: ab
  ! Enter exactly one letter.
Guess a letter:
  ! Enter exactly one letter.
Guess a letter: 1
  ! That is not a letter (A-Z only).
Guess a letter: ?
  ! That is not a letter (A-Z only).
Guess a letter: e
  Yes! 'E' appears 1 time(s).
Word : _ _ _ _ _ _ E _    Lives: 6   Missed:
Guess a letter: E
  ! You already tried that letter.
Guess a letter: F
   ...
```

H2 log at the end of the game

```
*** You got it! The word was SYSCALL. ***
Score: 1 won, 0 lost.
Play again? (Y/N): N
Thanks for playing!
```

Test summary

Every test ran on the MIPS program in MARS and on the Java program with the same input. 31 of 31 passed. The edge cases cover letters and blank lines where a number goes and numbers out of range and taken cells and bad commands. They also cover multi character and non letter guesses and repeated guesses and bad play again answers and the state resetting between games. For blackjack they cover soft aces and natural blackjack and the dealer's 17 rule and a bust ending the round before the dealer plays.

Test	Game	MIPS	Java
T1 X wins with the middle column	tictactoe	pass	pass
T2 O wins on the diagonal	tictactoe	pass	pass
T3 Full board with no winner	tictactoe	pass	pass
T4 Bad input. Text then a blank line then 0 then 10 then spaces around a number then a taken cell	tictactoe	pass	pass
T5 Bad play again answer and the board resets for a second game	tictactoe	pass	pass
B1 Stand right away and let the dealer play	blackjack	pass	pass
B2 Basic strategy. Hit under 17 and stand at 17 or more	blackjack	pass	pass
B3 Three rounds in a row and the win loss push counters add up	blackjack	pass	pass
B4 Bad commands (7 then a blank line then ?) are rejected without dealing a card	blackjack	pass	pass
B5 Hit until bust and then a bad play again answer	blackjack	pass	pass
H1 Solve the word with no misses	hangman	pass	pass

Test	Game	MIPS	Java
H2 Win after three misses so part of the figure is drawn	hangman	pass	pass
H3 Six misses so the game is lost and the word is shown	hangman	pass	pass
H4 Bad guesses. Two letters then a blank line then a digit then a symbol then a repeat (no life lost)	hangman	pass	pass
H5 Lower case works and a bad play again answer and a new word with 6 lives on replay	hangman	pass	pass
B6 40 round blackjack check	blackjack	pass	-

Source code

All of the code is on GitHub at github.com/dflo042/cs/tree/main/mips-game-project. The mips folder has tictactoe.asm and blackjack.asm and hangman.asm. The java folder has the Java planning versions TicTacToe.java and Blackjack.java and Hangman.java. The display_renders folder has the display images from my test runs. Each .asm file starts with a comment block that says how to run it and what the data structures are and the register conventions.

I wrote the Java versions first to work out the game logic and the display layout. Each static method became a MIPS procedure with the same name and the arrays and counters match the .data section. The Display class in each Java file is a model of the Bitmap Display with the same font and drawing routines. Porting was mostly turning methods into procedures and ifs and whiles into branches and jumps.

Sources and acknowledgments

- ZyBooks Chapter 4 for procedures and the stack and arrays and branching.
- The MARS help menu for the syscall numbers (4 5 8 10 11 30 40 42) and the Bitmap Display settings. MARS 4.5 is by Pete Sanderson and Ken Vollmar (dpetersanderson.github.io).
- The Fisher-Yates shuffle and the dealer's hit on 16 stand on 17 rule are the standard way to do those. No code was copied.
- For testing I ran the programs in MARS from the command line with scripted input. I used Python scripts to type the answers and dump the display memory to PNG files and double check the blackjack math. Java was OpenJDK 21.
- I used Claude (an AI assistant) to help write and debug the MIPS and Java code and the test scripts and this write up. I went through the code and ran all the tests and I am responsible for the submission.
- I did not work with other students on this.